RAJENDRA UNIVERSITY

Department of Computer Science

DATABASE MANAGEMENT SYSTEM

Complete Study Notes

(Unit I – IV: ER & EER Model, Relational Model & SQL,
Normalization, Transaction Processing & Concurrency Control)

Prepared by

MANTHAN SAHU

Student of Computer Science
Rajendra University

Table of Contents

Important Questions (1, 2, 5 & 8 Marks) — Unit-wise

Text Books & Reference Books

UNIT I — Introduction to Database & ER / EER Model

1.0 Database Systems vs Traditional File Systems

Before DBMSs became common, organisations stored data in flat files managed directly by application programs (the 'file-processing system'). This approach suffered from several drawbacks that motivated the development of database systems:

Aspect	File-Processing System	Database Management System
Data Redundancy	High — same data duplicated across many files used by different programs	Minimised through a centralized, shared data repository
Data Inconsistency	Likely, since updates may not be reflected everywhere	Reduced, since data is stored once and constraints are enforced
Data Access	Difficult — requires writing a new program for every new kind of query	Easy — flexible query languages such as SQL support ad-hoc queries
Data Isolation	Data scattered in various files of different formats	Data integrated in a single logical structure
Integrity Constraints	Hard-coded in application programs, easily bypassed	Centrally enforced by the DBMS (keys, domains, referential integrity)
Atomicity of Updates	Not guaranteed — a system crash may leave data inconsistent	Guaranteed through transaction management
Concurrent Access	Difficult to manage safely	Managed by the DBMS through concurrency control protocols
Security	Weak, difficult to enforce uniformly	Centralized authorization and access control

1.1 Introduction to Database and Database Users

A database is an organized, integrated collection of related data that is stored so that it can be efficiently accessed, managed, and updated. A Database Management System (DBMS) is the software that enables users to define, create, maintain, and control access to the database. Together, the database and DBMS software, along with associated applications, are referred to as a database system.

Why use a DBMS instead of files?

- Redundancy control — the same data is not duplicated across many files, saving storage and avoiding inconsistency.
- Restricting unauthorized access — DBMS provides security and authorization subsystems so that only permitted users can access or modify data.
- Providing persistent storage for program objects and data structures.
- Providing multiple interfaces to different classes of users (query languages, forms, APIs).

- Representing complex relationships among data using keys and constraints.
- Enforcing integrity constraints (domain, key, referential integrity).
- Providing backup and recovery services to protect against failures.
- Permitting inferencing and actions using rules and triggers.

Categories of Database Users

Users of a DBMS can broadly be divided into those who actually use and control the database content (actors on the scene) and those who design, develop and maintain the DBMS software (workers behind the scene).

(a) Actors on the Scene

- Database Administrators (DBA) — responsible for authorizing access, coordinating and monitoring use, acquiring hardware/software resources, and overall control of the database system.
- Database Designers — responsible for identifying the data to be stored and choosing appropriate structures to represent this data (conceptual, logical and physical design).
- End Users — the people whose jobs require access to the database for querying, updating and generating reports. These include casual end users, naive/parametric end users, sophisticated end users, and stand-alone users.
- System Analysts and Application Programmers — analysts determine the requirements of end users; application (software engineer) programmers implement these specifications as programs, then test, debug, document and maintain them.

(b) Workers Behind the Scene

- DBMS System Designers and Implementers — design and implement the DBMS modules and interfaces as a software package.
- Tool Developers — design and implement tools such as report writers, performance monitors, data-modelling packages.
- Operators and Maintenance Personnel — responsible for actually running and maintaining the hardware and software environment.

1.2 Database System Concepts and Architecture

Data Models, Schemas and Instances

A data model is a collection of concepts that can be used to describe the structure of a database — the data types, relationships and constraints. It usually also includes a set of basic operations for retrievals and updates. Data models can be classified into:

- High-level / conceptual data models — provide concepts close to the way users perceive data (e.g., the Entity-Relationship model).
- Representational (implementation) data models — provide concepts understood by end users but not too far from database organisation (e.g., relational data model).
- Low-level / physical data models — describe how data is stored as files, indexes, record ordering, and access paths.

A database schema is the overall description of the database, specified during database design and expected not to change frequently. A schema diagram displays only the schema constructs (not their instances). Each object in the schema is called a schema construct.

A database state or snapshot is the actual data stored in the database at a particular moment; it is also called the database instance. Every time the database is updated it moves from one state to another. The DBMS enforces that every state satisfies the structure and constraints specified in the schema — this is called the valid state.

The Three-Schema Architecture

Proposed by ANSI/SPARC, the three-schema architecture aims to separate the user applications from the physical database through three levels of abstraction:

- Internal level — has an internal schema describing the physical storage structure of the database (data files, indexing, storage format).
- Conceptual level — has a conceptual schema describing the structure of the whole database for the community of users, hiding physical storage details and concentrating on entities, data types, relationships and constraints.
- External (view) level — includes a number of external schemas or user views, each describing the part of the database that a particular user group is interested in, hiding the rest of the database.

Data Independence

Data independence is the capacity to change the schema at one level without having to change the schema at the next higher level.

- Logical Data Independence — the capacity to change the conceptual schema without having to change external schemas or application programs. Changes at the conceptual level (e.g., adding an entity) may be needed when the logical structure of the database is altered.
- Physical Data Independence — the capacity to change the internal schema without having to change the conceptual schema. Changes to physical storage or access methods do not require changes in application programs, only in the internal schema.

DBMS Languages

- Data Definition Language (DDL) — used by the DBA and database designers to specify the conceptual schema (e.g., CREATE, ALTER, DROP).
- Data Manipulation Language (DML) — used to specify database retrievals and updates (e.g., SELECT, INSERT, UPDATE, DELETE). High-level (declarative) DMLs such as SQL can specify and retrieve many records in a single statement and are called set-at-a-time or set-oriented; low-level or procedural DMLs retrieve records one at a time and are called record-at-a-time.
- Data Control Language (DCL) — used to control access and permissions (e.g., GRANT, REVOKE).
- Storage Definition Language (SDL) — used to specify the internal schema, often combined with DDL in modern systems.

1.1.1 Types of Data Models (Historical Perspective)

Model	Description
Hierarchical Model	Organises data in a tree-like structure with parent-child (1:N) relationships between record types; navigation is top-down. Simple but inflexible for many-to-many relationships (e.g., IMS).
Network Model	Represents data as a graph, allowing a record to have multiple parents (implemented via 'sets'); more flexible than hierarchical but complex to navigate (e.g., CODASYL DBTG model).
Relational Model	Represents data as a collection of tables (relations) with rows and columns; based on strong mathematical foundations (set theory, predicate logic); the dominant model today (e.g., MySQL, Oracle, PostgreSQL).
Object-Oriented Model	Represents data as objects, combining data and the operations (methods) that act on it, supporting inheritance and complex types; suited to CAD/multimedia applications.
NoSQL Models	Modern non-relational models (document, key-value, column-family, graph) designed for horizontal scalability and flexible/semi-structured schemas, e.g., MongoDB (document), Redis (key-value), Neo4j (graph).

1.2.1 DBMS Architectures

Architecture	Description
Centralized	All DBMS functionality, application programs and the user interface run on a single computer system; typical of early mainframe and single-user PC databases.
Two-Tier (Client-Server)	The client machine runs the application/user interface and forwards requests to a database server that runs the DBMS and manages the data; commonly used for LAN-based systems.
Three-Tier	Adds a middle 'application server' (business logic) tier between the client (presentation) and the database server (data), improving scalability, security and maintainability — the standard architecture for web applications.
Distributed	The database itself is stored on several computers (sites) connected by a network; the DBMS coordinates access so the distribution is transparent to users.

1.3 Conceptual Modeling and Database Design

Conceptual (semantic) data modelling provides concepts that closely match the way users perceive data, and is normally used during the requirements analysis and initial phase of database design. The Entity-Relationship (ER) model, developed by Peter Chen (1976), is the most widely used conceptual model. Database design typically proceeds through the following phases:

1. Requirements collection and analysis — determining data and functional requirements of the intended database users.
2. Conceptual design — producing a conceptual schema (ER diagram) that is DBMS-independent and describes entities, relationships and constraints.
3. Logical design (data model mapping) — mapping the conceptual schema into an implementation data model (e.g., relational model) understood by the chosen DBMS.
4. Physical design — deciding on file organisation, indexing, and storage structures to achieve good performance.

1.4 Entity–Relationship (ER) Model

The ER model describes data in terms of entities, their attributes and relationships among entities. It is used to construct a conceptual schema for a database, which is later mapped into an implementation data model such as the relational model.

1.4.1 Entities, Entity Types and Entity Sets

An entity is a 'thing' in the real world with an independent existence — it may be an object with a physical existence (a person, a car, a house) or an object with a conceptual existence (a company, a job, a university course).

Every entity has attributes — the particular properties that describe it. For example, an EMPLOYEE entity may have attributes Name, Age, Address, Salary and Job. A particular entity has a value for each of its attributes, so the attribute values form a major part of the data stored in the database.

An entity type defines a collection (set) of entities that have the same attributes. It is described by a name and a list of attributes. The entity set (or entity collection) is the collection of all entities of a particular entity type in the database at a given point in time.

1.4.2 Attributes and their Types

- Simple (atomic) attributes — cannot be divided further, e.g., Age, Sex.
- Composite attributes — can be divided into smaller sub-parts, e.g., Address can be split into Street, City, State and Pin-code; Name can be split into First-Name, Middle-Name, Last-Name.
- Single-valued attributes — have only a single value for a particular entity, e.g., Age.
- Multi-valued attributes — may have more than one value for an entity, e.g., a person may have several college degrees or several phone numbers. Shown in an ER diagram with a double ellipse.
- Stored attributes — values are physically stored in the database, e.g., Date-of-Birth.
- Derived attributes — value can be derived from other related attributes/entities, e.g., Age can be derived from Date-of-Birth and current date; shown with a dashed ellipse.
- Null values — used when an entity does not have an applicable value for an attribute, or the value is unknown (may be 'unknown but exists' or 'not applicable').

1.4.3 Keys

A key is an attribute or a set of attributes whose values uniquely identify each entity in an entity set.

- Super Key — a set of one or more attributes that, taken collectively, uniquely identifies an entity. It may contain extra attributes not necessary for uniqueness.
- Candidate Key — a minimal super key, i.e., a super key from which no attribute can be removed without destroying the uniqueness property.
- Primary Key — the candidate key chosen by the database designer to uniquely identify entities; it cannot have a NULL value.
- Alternate Key — the candidate keys that are not chosen as the primary key.
- Composite Key — a primary key composed of more than one attribute.
- Foreign Key — an attribute in one relation whose values are required to match values in the primary key of another (or the same) relation; used to represent relationships between tables.

1.4.4 Relationship Types, Sets, Roles and Structural Constraints

A relationship type R among n entity types defines a set of associations (relationship set) among entities from these types. Each relationship instance in the relationship set is an association of entities, one entity from each participating entity type.

Degree of a Relationship

- Unary (Degree 1) — a relationship of an entity type to itself, e.g., an EMPLOYEE 'supervises' another EMPLOYEE (recursive relationship).
- Binary (Degree 2) — the most common type, involves two entity types, e.g., EMPLOYEE 'works_for' DEPARTMENT.
- Ternary (Degree 3) — involves three entity types, e.g., SUPPLIER 'supplies' PART to PROJECT.

Role Names

Each entity type participating in a relationship type plays a particular role. Role names are especially important for recursive relationship types to specify how an entity participates twice (or more) in a relationship instance. For example, in the SUPERVISION relationship, an EMPLOYEE entity plays the role of 'supervisor' and another plays the role of 'supervisee'.

Structural Constraints — Cardinality Ratio

The cardinality ratio specifies the maximum number of relationship instances that an entity can participate in.

Type	Description	Example
1:1 (One-to-One)	An entity in A is associated with at most one entity in B, and vice versa.	EMPLOYEE manages DEPARTMENT (a department has one manager, a manager manages one department)
1:N (One-to-Many)	An entity in A can be associated with many entities in B, but an entity in B is associated with only one entity in A.	DEPARTMENT has many EMPLOYEEs, an employee works for one department
M:N (Many-to-Many)	An entity in A can be associated with many entities in B and vice versa.	EMPLOYEE works on many PROJECTs; a PROJECT has many EMPLOYEEs

Participation Constraints

- Total Participation (existence dependency) — every entity in the entity set must participate in at least one relationship instance; shown with a double line in an ER diagram. For example, every EMPLOYEE must work for a DEPARTMENT.
- Partial Participation — only some entities of the entity set participate in the relationship; shown with a single line. For example, not every employee manages a department.

1.4.5 Weak Entity Types

Entity types that do not have key attributes of their own are called weak entity types; entity types that do have a key attribute are called strong (regular) entity types. A weak entity type is identified by being existence-dependent on a specific strong entity type, called its identifying/owner entity type, through a relationship called the identifying relationship.

- Entities belonging to a weak entity type are identified by the combination of a partial key of the weak entity and the primary key of its owner entity — this is called the partial key or discriminator.
- A weak entity type always has a total participation constraint with respect to its identifying relationship.
- In ER diagrams, a weak entity type is shown with a double rectangle, its partial key with a dashed underline, and the identifying relationship with a double diamond.

Example: A DEPENDENT entity (of an employee) is a weak entity — its partial key Name, combined with the primary key of EMPLOYEE, uniquely identifies a dependent.

1.4.6 ER Diagram Notations

Symbol	Meaning
Rectangle	Represents an entity type
Double Rectangle	Represents a weak entity type
Ellipse	Represents an attribute
Double Ellipse	Represents a multi-valued attribute
Dashed Ellipse	Represents a derived attribute
Underlined text	Represents the key attribute
Diamond	Represents a relationship type
Double Diamond	Represents an identifying relationship for a weak entity
Line	Links attributes to entity types and entity types to relationships
Double Line	Represents total participation of an entity in a relationship

1.4.7 ER Naming Conventions

- Entity type names are singular nouns, written in uppercase or with an initial capital letter, e.g., EMPLOYEE, CAR.

- Relationship type names are verbs or verb phrases, e.g., WORKS_FOR, MANAGES.
- Attribute names are nouns, generally starting with a lowercase letter, e.g., Name, Salary.
- Consistent naming avoids ambiguity and makes the diagram self-documenting for other designers.

1.5 Enhanced Entity–Relationship (EER) Model

The basic ER model is not sufficient to represent the requirements of more complex applications. The Enhanced ER (EER) model includes additional semantic concepts: specialization, generalization, categories (union types), and attribute/relationship inheritance.

1.5.1 Specialization

Specialization is the process of defining a set of subclasses of an entity type (called the superclass). It is a top-down conceptual refinement process. Each subclass entity is also a member of the superclass, and inherits all the attributes and relationships of the superclass — this is called attribute/relationship inheritance.

Example: The entity type EMPLOYEE may be specialized into subclasses SECRETARY, ENGINEER and TECHNICIAN based on job type.

1.5.2 Generalization

Generalization is the reverse of specialization — it is the process of defining a generalized (superclass) entity type from two or more entity types that represent similar lower-level (subclass) entities. It is a bottom-up conceptual synthesis process. For example, CAR and TRUCK may be generalized into VEHICLE.

1.5.3 Constraints on Specialization/Generalization

(a) Disjointness Constraint

- Disjoint (d) — an entity can be a member of at most one of the subclasses; subclasses are mutually exclusive, e.g., an employee is either SECRETARY, ENGINEER or TECHNICIAN, not more than one.
- Overlapping (o) — the same entity may be a member of more than one subclass, e.g., a person can be both a STUDENT and an EMPLOYEE.

(b) Completeness Constraint

- Total specialization — every entity in the superclass must be a member of at least one subclass; shown with a double line from superclass to circle.
- Partial specialization — an entity may not belong to any of the subclasses; shown with a single line.

Combining the two constraints gives four categories: disjoint total, disjoint partial, overlapping total and overlapping partial.

1.5.4 Union Type / Category

A category (union type) is a subclass of the union of two or more superclasses that can have different entity types (unlike specialization, where the subclass is related to a single superclass). For example, a database may contain OWNER entity type which could be a subset of the union of PERSON, BANK and COMPANY entity types — an owner of a registered vehicle may be a person, a bank (holding a lien) or a company.

1.5.5 Aggregation

Aggregation is an abstraction concept used to model a relationship between a whole object and its component parts, or to treat a relationship itself as a higher-level entity so that it can participate in further relationships. For example, the relationship WORKS_ON (between EMPLOYEE and PROJECT) can be aggregated so that it can further participate in a relationship MANAGES with a MANAGER entity.

1.6 Mapping the ER Model to the Relational Model

Once a conceptual ER schema is designed, it must be mapped (translated) into a set of relations for implementation. The standard algorithm consists of the following steps:

5. Step 1 — Mapping of Regular (Strong) Entity Types: for each regular entity type E, create a relation R that includes all simple attributes of E; for composite attributes, include only their simple component attributes; choose one candidate key of E as the primary key of R.
6. Step 2 — Mapping of Weak Entity Types: for each weak entity type W with owner entity type E, create a relation R that includes all simple attributes of W, plus the primary key of the owner entity's relation as a foreign key; the primary key of R is the combination of the owner's primary key and the partial key of W.
7. Step 3 — Mapping of Binary 1:1 Relationship Types: choose one of the two participating relations (preferably the one with total participation) and include the primary key of the other relation as a foreign key.
8. Step 4 — Mapping of Binary 1:N Relationship Types: include the primary key of the relation on the '1' side as a foreign key in the relation on the 'N' side.
9. Step 5 — Mapping of Binary M:N Relationship Types: create a new relation to represent the relationship; include as foreign keys the primary keys of both participating entity relations — together they typically form the primary key of the new relation; include any relationship attributes as well.
10. Step 6 — Mapping of Multivalued Attributes: for each multivalued attribute, create a new relation containing an attribute corresponding to the multivalued attribute, plus the primary key of the owning entity's relation as a foreign key.
11. Step 7 — Mapping of N-ary Relationship Types (degree > 2): create a new relation to represent the relationship, and include as foreign keys the primary keys of all participating entity relations, plus any relationship attributes.

1.7 Worked Example — ER Diagram for a University Database

Consider a simplified University Database that must record information about students, courses, departments and instructors. The conceptual design (before drawing the diagram) identifies the following entities, attributes and relationships:

Entities and Attributes

Entity	Key Attribute (underlined)	Other Attributes
STUDENT	Student_Id	Name (composite: First, Last), DOB, Address, Phone (multi-valued)

Entity	Key Attribute (underlined)	Other Attributes
COURSE	Course_Id	Course_Name, Credit_Hours
DEPARTMENT	Dept_No	Dept_Name, Location
INSTRUCTOR	Instructor_Id	Name, Qualification, Age (derived from DOB)

Relationships and Cardinalities

Relationship	Participating Entities	Cardinality	Participation
ENROLLS_IN	STUDENT, COURSE	M : N	STUDENT: partial, COURSE: total
BELONGS_TO	STUDENT, DEPARTMENT	N : 1	STUDENT: total, DEPARTMENT: partial
OFFERS	DEPARTMENT, COURSE	1 : N	COURSE: total, DEPARTMENT: partial
TEACHES	INSTRUCTOR, COURSE	1 : N	COURSE: total, INSTRUCTOR: partial
WORKS_IN	INSTRUCTOR, DEPARTMENT	N : 1	INSTRUCTOR: total, DEPARTMENT: partial

Reading the diagram: rectangles STUDENT, COURSE, DEPARTMENT and INSTRUCTOR are connected through diamond-shaped relationships ENROLLS_IN, BELONGS_TO, OFFERS, TEACHES and WORKS_IN. The ENROLLS_IN relationship between STUDENT and COURSE is many-to-many, so in the relational mapping it becomes its own table carrying both foreign keys (Student_Id, Course_Id) plus any relationship attribute such as Grade. All other relationships are one-to-many and are implemented by placing a foreign key on the 'many' side (e.g., Dept_No as a foreign key inside STUDENT and INSTRUCTOR).

UNIT II — Relational Data Model & SQL

2.1 Relational Model Concepts

The relational model represents the database as a collection of relations (tables). Informally, a relation looks like a table of values; each row represents a fact corresponding to a real-world entity or relationship.

Formal Term	Informal Equivalent
Relation	Table
Tuple	Row / Record
Attribute	Column / Field
Domain	Set of legal (permitted) values for an attribute
Degree	Number of attributes (columns) in a relation
Cardinality	Number of tuples (rows) in a relation
Relation Schema	R(A1, A2, ..., An) — the name of the relation and its attributes
Relation State / Instance	The set of tuples currently in the relation

Properties of Relations

- Each cell of a relation contains exactly one atomic (single) value.
- Each attribute has a distinct name.
- Values of an attribute are all from the same domain.
- Each tuple is distinct — there are no duplicate tuples in a relation.
- The order of attributes and the order of tuples has no significance (theoretically).

Relational Integrity Constraints

- Domain Constraint — every value of an attribute must be atomic and drawn from the domain of that attribute.
- Key Constraint — every relation must have a primary key; no two tuples can have the same combination of values for all the primary key attributes.
- Entity Integrity Constraint — no attribute of the primary key can have a NULL value, since the primary key is used to identify individual tuples.
- Referential Integrity Constraint — this is specified between two relations and is used to maintain the consistency among tuples of the two relations. A tuple in one relation that refers to another relation must refer to an existing tuple in that relation (enforced through foreign keys).

2.2 SQL — Structured Query Language

SQL is a comprehensive database language — it has statements for data definition, query, update and view/access control. It is a declarative (non-procedural) language: the user specifies what data is needed rather than how to retrieve it.

2.2.1 Categories of SQL commands

Category	Purpose	Commands
DDL (Data Definition Language)	Define / modify database structure	CREATE, ALTER, DROP, TRUNCATE
DML (Data Manipulation Language)	Manipulate data stored in tables	SELECT, INSERT, UPDATE, DELETE
DCL (Data Control Language)	Control access / permissions	GRANT, REVOKE
TCL (Transaction Control Language)	Manage transactions	COMMIT, ROLLBACK, SAVEPOINT

2.2.2 SQL Data Types

- Numeric — INT, SMALLINT, DECIMAL(p,q), FLOAT, DOUBLE
- Character/String — CHAR(n) (fixed length), VARCHAR(n) (variable length)
- Date/Time — DATE, TIME, TIMESTAMP
- Boolean — BOOLEAN (TRUE / FALSE)

2.2.3 Data Definition — CREATE, ALTER, DROP

Syntax to create a table:

```
CREATE TABLE EMPLOYEE (
    Emp_Id      INT PRIMARY KEY,
    Name        VARCHAR(30) NOT NULL,
    Dept_No     INT,
    Salary      DECIMAL(10,2) CHECK (Salary > 0),
    Dept_No     INT REFERENCES DEPARTMENT(Dept_No)
);
```

Common constraints usable in CREATE TABLE:

Constraint	Purpose
PRIMARY KEY	Uniquely identifies each row; implies NOT NULL and UNIQUE
FOREIGN KEY / REFERENCES	Enforces referential integrity with another table
NOT NULL	Column cannot store a NULL value

Constraint	Purpose
UNIQUE	All values in the column must be distinct
CHECK	Restricts values based on a logical condition
DEFAULT	Assigns a default value when none is specified

Modifying and removing tables:

```
ALTER TABLE EMPLOYEE ADD Email VARCHAR(50);
ALTER TABLE EMPLOYEE MODIFY Salary DECIMAL(12,2);
ALTER TABLE EMPLOYEE DROP COLUMN Email;
DROP TABLE EMPLOYEE;
TRUNCATE TABLE EMPLOYEE;   -- removes all rows, keeps structure
```

2.2.4 Data Manipulation — INSERT, UPDATE, DELETE

```
-- INSERT
INSERT INTO EMPLOYEE (Emp_Id, Name, Dept_No, Salary)
VALUES (101, 'Manthan Sahu', 3, 45000.00);

-- UPDATE
UPDATE EMPLOYEE
SET Salary = Salary * 1.10
WHERE Dept_No = 3;

-- DELETE
DELETE FROM EMPLOYEE
WHERE Emp_Id = 101;
```

2.2.5 Operators used in the WHERE clause

Category	Operators
Comparison	= , <> (or !=), < , > , <= , >=
Logical	AND, OR, NOT
Range	BETWEEN … AND …
Set Membership	IN, NOT IN
Pattern Matching	LIKE with wildcards % (any sequence) and _ (any single character)
Null Check	IS NULL, IS NOT NULL

2.2.6 Retrieval Queries — SELECT

General syntax:

```
SELECT [DISTINCT] column_list
FROM table_list
[WHERE condition]
[GROUP BY column_list]
[HAVING group_condition]
[ORDER BY column_list [ASC | DESC]];
```

Examples:

```
-- 1. Retrieve all columns
SELECT * FROM EMPLOYEE;

-- 2. Retrieve with condition
SELECT Name, Salary FROM EMPLOYEE WHERE Dept_No = 3;

-- 3. Sorting results
SELECT Name, Salary FROM EMPLOYEE ORDER BY Salary DESC;

-- 4. Aggregate functions with grouping
SELECT Dept_No, AVG(Salary) AS Avg_Salary
FROM EMPLOYEE
GROUP BY Dept_No
HAVING AVG(Salary) > 30000;

-- 5. Pattern matching
SELECT * FROM EMPLOYEE WHERE Name LIKE 'M%';

-- 6. Set membership
SELECT * FROM EMPLOYEE WHERE Dept_No IN (2, 3, 5);

-- 7. Nested (sub) query
SELECT Name FROM EMPLOYEE
WHERE Salary > (SELECT AVG(Salary) FROM EMPLOYEE);
```

2.2.7 Built-in (Aggregate) Functions

Function	Purpose
COUNT()	Counts the number of tuples/values
SUM()	Sums numeric values in a column
AVG()	Computes the average value

Function	Purpose
MAX()	Finds the maximum value
MIN()	Finds the minimum value

2.2.8 Joins in SQL

Join Type	Description
INNER JOIN	Returns only matching rows from both tables
LEFT (OUTER) JOIN	Returns all rows of the left table + matching rows from right; NULL where no match
RIGHT (OUTER) JOIN	Returns all rows of the right table + matching rows from left; NULL where no match
FULL (OUTER) JOIN	Returns all rows from both tables, with NULLs where no match exists
SELF JOIN	A table is joined with itself, typically using aliases

```
SELECT E.Name, D.Dept_Name
FROM EMPLOYEE E
INNER JOIN DEPARTMENT D ON E.Dept_No = D.Dept_No;
```

2.2.9 Data Control Language

```
GRANT SELECT, INSERT ON EMPLOYEE TO user1;
REVOKE INSERT ON EMPLOYEE FROM user1;
```

2.2.10 Commonly Used String and Date Functions

Function	Purpose
UPPER() / LOWER()	Converts a string to upper-case / lower-case
LENGTH() / LEN()	Returns the number of characters in a string
SUBSTR() / SUBSTRING()	Extracts a portion of a string
CONCAT()	Joins two or more strings together
TRIM()	Removes leading/trailing spaces
ROUND()	Rounds a numeric value to a given number of decimal places
SYSDATE / NOW()	Returns the current system date and time
DATEDIFF()	Returns the difference between two dates

2.2.11 Indexes

An index is an auxiliary access structure built on one or more columns of a table to speed up retrieval of rows that satisfy certain conditions, at the cost of extra storage and slower updates. The primary key of a table is automatically indexed by most DBMSs.

```
CREATE INDEX idx_emp_dept ON EMPLOYEE (Dept_No);
DROP INDEX idx_emp_dept;
```

2.2.12 Additional Solved SQL Queries

Using EMPLOYEE(Emp_Id, Name, Dept_No, Salary) and DEPARTMENT(Dept_No, Dept_Name, Location):

```
-- 1. Employees with no department assigned
SELECT Name FROM EMPLOYEE WHERE Dept_No IS NULL;

-- 2. Second highest salary
SELECT MAX(Salary) FROM EMPLOYEE
WHERE Salary < (SELECT MAX(Salary) FROM EMPLOYEE);

-- 3. Departments having more than 5 employees
SELECT Dept_No, COUNT(*) AS Emp_Count
FROM EMPLOYEE GROUP BY Dept_No HAVING COUNT(*) > 5;

-- 4. Employees whose name starts with 'S' and ends with 'a'
SELECT Name FROM EMPLOYEE WHERE Name LIKE 'S%a';

-- 5. Departments with no employees (LEFT JOIN + IS NULL)
SELECT D.Dept_Name FROM DEPARTMENT D
LEFT JOIN EMPLOYEE E ON D.Dept_No = E.Dept_No
WHERE E.Emp_Id IS NULL;
```

2.3 Relational Algebra

Relational algebra is a procedural query language: it takes relations as input and produces a new relation as output, using a set of operations. It provides the formal foundation for relational database operations and for SQL query optimisation.

2.3.1 Unary Relational Operations

SELECT (σ) Operation

The SELECT operation is used to select a subset of tuples from a relation that satisfy a given predicate (selection condition). It works on a single relation R and is denoted:

```
σ <selection_condition> (R)
Example:  σ Dept_No = 3 (EMPLOYEE)
```

The SELECT operation is commutative, and its result has the same degree (number of attributes) as R, but with (generally) fewer tuples.

PROJECT (π) Operation

The PROJECT operation selects certain columns (attributes) from a relation and discards the other columns; it also removes duplicate tuples from the result (since a relation is a set). Denoted:

```
π <attribute_list> (R)
Example:  π Name, Salary (EMPLOYEE)
```

The number of tuples in the result of PROJECT is always less than or equal to the number of tuples in R; the PROJECT operation is not commutative in general (order of attribute list matters for display, though the underlying set does not).

RENAME (ρ) Operation

Used to rename either the relation itself or its attributes, denoted ρ S(B1,B2,…,Bn) (R), useful for self-joins and building clearer expressions.

2.3.2 Set Operations

- UNION (∪) — combines tuples from two union-compatible relations, removing duplicates.
- INTERSECTION (∩) — retains tuples common to both relations.
- SET DIFFERENCE (−) — retains tuples present in the first relation but not the second.
- CARTESIAN PRODUCT (×) — combines every tuple of R1 with every tuple of R2.

2.3.3 Binary Relational Operations — JOIN and DIVISION

JOIN (⋈) Operation

The JOIN operation is used to combine related tuples from two relations into single tuples, based on a join condition. It is essentially a Cartesian product followed by a selection.

- Theta Join (θ-join) — general join with any condition R ⋈θ S.
- Equi-join — a theta join where the condition consists only of equality comparisons.
- Natural Join (⋈) — an equi-join on all common attribute names, with the duplicate attribute automatically removed from the result.

```
EMPLOYEE ⋈ Dept_No=Dept_No DEPARTMENT
-- Natural Join equivalent:
EMPLOYEE ⋈ DEPARTMENT
```

DIVISION (÷) Operation

The DIVISION operation, denoted R ÷ S, is applicable when R(Z) has attributes Z = X ∪ Y and S(Y) has attributes Y. It returns tuples over X such that, for every tuple in S, the combination of the X-tuple with the Y-tuple exists in R. It is useful for queries containing the phrase 'for all', for example: 'find employees who work on all projects that department 5 controls'.

2.3.4 Relational Calculus

Relational calculus is a non-procedural (declarative) query language equivalent in expressive power to relational algebra — it describes what is to be retrieved, not how, using formal predicate logic. It is not used directly by end users but provides the theoretical basis for languages like SQL. There are two forms:

Tuple Relational Calculus (TRC)

A query in TRC has the general form { t | P(t) }, meaning 'the set of all tuples t such that predicate P(t) is true', where t is a tuple variable ranging over a relation.

```
-- Find names of employees in department 5
{ t.Name | EMPLOYEE(t) AND t.Dept_No = 5 }
```

Domain Relational Calculus (DRC)

DRC uses domain variables that range over the individual values (domains) of attributes rather than whole tuples. A query has the general form { <x1, x2, ..., xn> | P(x1, x2, ..., xn) }.

```
-- Find names of employees in department 5
{ <n> | ∃ id, d, s ( EMPLOYEE(id, n, d, s) AND d = 5 ) }
```

Both forms use quantifiers — the existential quantifier ∃ ('there exists') and the universal quantifier ∀ ('for all') — to bind variables. Relational calculus is said to be 'safe' when it is guaranteed to yield a finite result; SQL's SELECT-FROM-WHERE syntax is essentially a safe, user-friendly rendering of tuple relational calculus.

2.4 Sub-queries, Views and Additional SQL Features

2.4.1 Types of Sub-queries (Nested Queries)

- Single-row sub-query — returns exactly one row and is used with comparison operators (=, <, >, etc.).
- Multiple-row sub-query — returns more than one row and must be used with IN, ANY, ALL or EXISTS.
- Correlated sub-query — the inner query depends on (references a column from) the outer query, and is re-evaluated once for every row processed by the outer query.
- EXISTS / NOT EXISTS — tests for the existence (or non-existence) of any row returned by a sub-query, commonly used for correlated sub-queries.

```
-- Multiple-row subquery with ANY
```

```
SELECT Name FROM EMPLOYEE
WHERE Salary > ANY (SELECT Salary FROM EMPLOYEE WHERE Dept_No = 5);

-- Correlated subquery with EXISTS
SELECT Name FROM EMPLOYEE E
WHERE EXISTS (SELECT * FROM DEPENDENT D WHERE D.Emp_Id = E.Emp_Id);
```

2.4.2 Set Operators in SQL

```
SELECT Name FROM EMPLOYEE WHERE Dept_No = 3
UNION
SELECT Name FROM EMPLOYEE WHERE Salary > 40000;

-- INTERSECT returns rows common to both queries
-- EXCEPT (MINUS in Oracle) returns rows in the first query not in the second
```

2.4.3 Views

A view is a virtual table derived from one or more base tables (or other views) through a stored SQL query; it does not store data itself but presents the result of the defining query whenever it is accessed. Views are used to simplify complex queries, provide a security/authorization mechanism by restricting visible columns/rows, and support logical data independence.

```
CREATE VIEW DeptSalary AS
SELECT Dept_No, AVG(Salary) AS Avg_Salary
FROM EMPLOYEE
GROUP BY Dept_No;

SELECT * FROM DeptSalary WHERE Avg_Salary > 35000;
```

2.4.4 Worked Example — Sample Tables and a Join Query

Consider two small sample tables:

Emp_Id	Name	Dept_No
101	Aarav	1
102	Riya	2
103	Kabir	1

Dept_No	Dept_Name
1	Computer Science

Dept_No	Dept_Name
2	Electronics

The query below performs an inner join and returns each employee alongside their department name:

```
SELECT E.Name, D.Dept_Name
FROM EMPLOYEE E JOIN DEPARTMENT D ON E.Dept_No = D.Dept_No;
```

Name	Dept_Name
Aarav	Computer Science
Riya	Electronics
Kabir	Computer Science

UNIT III — Database Design Theory & Normalization

3.1 Why Normalization?

Normalization is the process of organising data in a database to reduce redundancy and avoid undesirable characteristics like Insertion, Update and Deletion anomalies. It is a step-by-step formal process that decomposes 'bad' relation schemas (having redundant/repeated data) into smaller, well-structured relations.

Update Anomalies caused by redundancy

- Insertion Anomaly — inability to add data to the database because of the absence of other data, e.g., cannot insert a new employee unless they are assigned to a project.
- Deletion Anomaly — unintended loss of data due to deletion of other data, e.g., deleting the only employee of a department also deletes information about the department itself.
- Modification (Update) Anomaly — a change in one attribute requires multiple rows of data to be changed, otherwise data becomes inconsistent, e.g., changing a department's location requires updating it in every tuple of every employee in that department.

3.2 Functional Dependencies (FD)

A functional dependency, denoted X → Y, between two sets of attributes X and Y that are both subsets of R specifies a constraint on the possible tuples that can form a relation state r of R: for any two tuples t1 and t2 in r that have t1[X] = t2[X], we must also have t1[Y] = t2[Y]. This means the value of X uniquely (functionally) determines the value of Y; equivalently, Y is functionally dependent on X.

A functional dependency is a property of the semantics (meaning) of the attributes in the relation schema R, determined by the real-world constraints that the data must satisfy — it cannot be inferred automatically, but must be specified by someone who knows the semantics of the attributes.

Types of Functional Dependency

- Trivial FD — X → Y where Y is a subset of X, e.g., {Emp_Id, Name} → Emp_Id.
- Non-trivial FD — X → Y where Y is not a subset of X.
- Full functional dependency — Y is fully functionally dependent on X if it is functionally dependent on X, but not on any proper subset of X (relevant when X is a composite attribute).
- Partial dependency — an attribute is functionally dependent on only part of a composite primary key (rather than the whole key).
- Transitive dependency — a condition where X → Y and Y → Z, so X → Z indirectly (X does not directly determine Z).

Armstrong's Axioms (Inference Rules for FDs)

Rule	Statement
Reflexivity	If Y ⊆ X, then X → Y

Rule	Statement
Augmentation	If X → Y, then XZ → YZ
Transitivity	If X → Y and Y → Z, then X → Z
Union (Additivity)	If X → Y and X → Z, then X → YZ
Decomposition	If X → YZ, then X → Y and X → Z
Pseudo-transitivity	If X → Y and WY → Z, then WX → Z

3.2.1 Closure of a Set of Functional Dependencies (F+)

The closure of a set of functional dependencies F, written F+, is the set of all functional dependencies that can be logically inferred from F using Armstrong's axioms. Related to this is the attribute closure X+ of an attribute set X under F — the set of all attributes that are functionally determined by X. X+ is computed by repeatedly adding to X any attribute A for which some FD Y → A holds with Y ⊆ X (current closure), until no further attributes can be added. X is a super key of R if and only if X+ contains all attributes of R.

3.2.2 Minimal (Canonical) Cover

A minimal cover Fc of a set of functional dependencies F is an equivalent set of FDs (F+ = Fc+) such that: (i) every right-hand side of every FD in Fc is a single attribute, (ii) no FD in Fc contains an extraneous attribute on the left-hand side, and (iii) no FD in Fc can be removed without changing the closure. Minimal covers are used to determine an optimal (least redundant) set of dependencies before designing relations, and are essential input to the 3NF synthesis algorithm.

3.2.3 Properties of Relational Decomposition

- Lossless-Join (Non-additive Join) Decomposition — a decomposition of R into R1 and R2 is lossless if joining R1 and R2 back together (natural join) reproduces exactly the original relation R, with no spurious tuples. This holds if and only if the common attributes (R1 ∩ R2) form a super key of at least one of R1 or R2.
- Dependency-Preserving Decomposition — a decomposition preserves all the functional dependencies of the original relation, meaning each FD in F can be verified using only the FDs that hold within the individual decomposed relations, without needing to join them back together. This allows integrity constraints to be checked efficiently without extra joins.

Every decomposition into BCNF is guaranteed to be lossless-join, but is not always dependency-preserving; every decomposition into 3NF (via the standard synthesis algorithm) can always be made both lossless-join and dependency-preserving.

3.3 Normal Forms Based on Primary Keys

3.3.1 First Normal Form (1NF)

A relation is in 1NF if every attribute value in it is atomic (single, indivisible value) — that is, the relation contains no composite or multi-valued attributes, and no repeating groups. 1NF disallows nested relations and multivalued/composite attributes.

Example: an EMPLOYEE relation with an attribute Phone_Numbers holding multiple values '9876543210, 9123456780' for a single employee violates 1NF. To normalize, either create a separate row for each phone number or move phone numbers into a new relation EMP_PHONE(Emp_Id, Phone_Number).

3.3.2 Second Normal Form (2NF)

A relation is in 2NF if it is already in 1NF and every non-prime attribute (an attribute not part of any candidate key) is fully functionally dependent on the primary key — i.e., there is no partial dependency of any non-prime attribute on a proper subset of the primary key. 2NF applies only when the primary key is composite.

Example: Consider STUDENT_COURSE(Student_Id, Course_Id, Student_Name, Course_Name, Grade), where the primary key is (Student_Id, Course_Id). Here, Student_Name depends only on Student_Id and Course_Name depends only on Course_Id — both are partial dependencies. To convert to 2NF, decompose into:

```
STUDENT(Student_Id, Student_Name)
COURSE(Course_Id, Course_Name)
ENROLLMENT(Student_Id, Course_Id, Grade)
```

3.3.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key — i.e., for every non-trivial FD X → A, either X is a super key, or A is a prime attribute (part of some candidate key).

Example: Consider EMPLOYEE(Emp_Id, Dept_No, Dept_Name), where Emp_Id → Dept_No and Dept_No → Dept_Name, so Emp_Id → Dept_Name transitively. To convert to 3NF, decompose into:

```
EMPLOYEE(Emp_Id, Dept_No)
DEPARTMENT(Dept_No, Dept_Name)
```

3.3.4 Boyce–Codd Normal Form (BCNF)

A relation is in BCNF if, for every non-trivial functional dependency X → Y, X must be a super key of the relation. BCNF is a stricter version of 3NF — it removes the exception that allowed A to be a prime attribute. Every relation in BCNF is also in 3NF, but not every 3NF relation is in BCNF (violations can occur when there are multiple overlapping candidate keys).

Example: Consider a relation STUDENT_ADVISOR(Student_Id, Subject, Advisor), where each advisor teaches only one subject, but a subject may be taught by several advisors, and Advisor → Subject holds although

Advisor is not a super key — this violates BCNF even though it may be in 3NF. Decomposing into ADVISOR_SUBJECT(Advisor, Subject) and STUDENT_ADVISOR(Student_Id, Advisor) restores BCNF.

Comparison: 3NF vs BCNF

Aspect	3NF	BCNF
Condition	X→A: X is a super key OR A is a prime attribute	X→A: X must always be a super key
Redundancy	May still contain some redundancy	Removes virtually all redundancy based on FDs
Dependency Preservation	Always preserves all functional dependencies	Decomposition may not preserve all FDs
Strictness	Less strict	Stricter form of 3NF

3.4 Multivalued Dependency and Fourth Normal Form (4NF)

A multivalued dependency (MVD) X →→ Y is said to hold on relation R if, for a given value of X, there exists a set of values for Y, and this set is independent of the values in the other attributes Z = R − X − Y. MVDs arise when two or more independent multivalued facts about the same entity are stored in a single relation, leading to redundancy.

A relation is in 4NF if it is in BCNF and has no non-trivial multivalued dependency — i.e., for every non-trivial MVD X →→ Y, X must be a super key.

Classic example: EMP_SKILLS_DEPENDENTS(Emp_Id, Skill, Dependent_Name) — an employee may have multiple skills and multiple dependents, and these two facts are independent of each other, causing redundant combinations. Decomposing into EMP_SKILLS(Emp_Id, Skill) and EMP_DEPENDENTS(Emp_Id, Dependent_Name) removes the anomaly and achieves 4NF.

3.5 Join Dependency and Fifth Normal Form (5NF)

A join dependency (JD), denoted *(R1, R2, ..., Rn), exists on relation R if R1, R2, ..., Rn form a lossless-join decomposition of R — that is, joining R1, R2, ..., Rn (via natural join) reconstructs R exactly, with no spurious tuples.

A relation is in 5NF (also called Project-Join Normal Form, PJNF) if it is in 4NF and every join dependency in it is implied by the candidate keys — that is, the relation cannot be further decomposed into smaller relations without loss of information, unless the decomposition follows from the candidate keys. 5NF deals with cases where information cannot be reconstructed from smaller decomposed tables using only binary joins, but requires three or more relations to be joined back together (a scenario found in certain many-to-many-to-many relationships).

Summary Table of Normal Forms

Normal Form	Rule Removed	Condition to Satisfy
1NF	Repeating groups / non-atomic values	All attribute values must be atomic
2NF	Partial dependency	1NF + no partial dependency on the primary key
3NF	Transitive dependency	2NF + no transitive dependency on the primary key
BCNF	Anomalies from overlapping candidate keys	For every FD X→Y, X must be a super key
4NF	Multivalued dependency	BCNF + no non-trivial MVD unless X is a super key
5NF	Join dependency	4NF + every join dependency is implied by candidate keys

3.3.5 Worked Example — Decomposing a 3NF Relation into BCNF

Consider a relation TEACH(Student, Course, Instructor) with the following functional dependencies, based on the rule that each instructor teaches only one course, but a course may be taught by several instructors:

```
FD1:  {Student, Course} -> Instructor
FD2:  Instructor -> Course
```

The only candidate key of TEACH is {Student, Course} (since Student and Course together determine Instructor, and no smaller set does). Checking BCNF: for FD2 (Instructor → Course), the left-hand side 'Instructor' is NOT a super key of TEACH — this violates BCNF, even though TEACH is already in 3NF (Course is a prime attribute, satisfying the 3NF exception clause).

To achieve BCNF, decompose TEACH based on the violating FD (Instructor → Course):

```
R1 (Instructor, Course)     -- derived from the violating FD
R2 (Student, Instructor)    -- remaining attributes plus the determinant
```

Both R1 and R2 are now in BCNF: in R1, Instructor is a super key (Instructor → Course); in R2, {Student, Instructor} is the only key and there are no other non-trivial FDs. Note, however, that this decomposition is lossless-join (since Instructor, the common attribute, is a key of R1) but is NOT dependency-preserving, because FD1 ({Student, Course} → Instructor) cannot be verified from R1 and R2 alone without rejoining them — this is a well-known trade-off when converting a 3NF schema to BCNF.

3.6 Worked Example — Step-by-Step Normalization

Consider the following unnormalized ORDER_DETAILS relation used by a company to record customer orders:

```
ORDER_DETAILS (Order_Id, Cust_Id, Cust_Name, Cust_City,
               {Product_Id, Product_Name, Unit_Price, Quantity})
```

Here the attributes in braces repeat for every product in an order, so the relation is not in 1NF.

Step 1 — Convert to 1NF

Remove the repeating group by allowing one row per (Order_Id, Product_Id) combination instead of nesting multiple products inside a single order row:

```
ORDER_DETAILS_1NF (Order_Id, Cust_Id, Cust_Name, Cust_City,
                   Product_Id, Product_Name, Unit_Price, Quantity)
```

Assume the primary key of this 1NF relation is the composite key (Order_Id, Product_Id). The following functional dependencies hold:

```
Order_Id                -> Cust_Id, Cust_Name, Cust_City
Cust_Id                 -> Cust_Name, Cust_City   (transitive, via Order_Id)
Product_Id               -> Product_Name, Unit_Price
(Order_Id, Product_Id)   -> Quantity
```

Step 2 — Convert to 2NF

Cust_Name, Cust_City, Product_Name and Unit_Price each depend on only part of the composite key — this is partial dependency. Remove it by decomposing:

```
ORDERS       (Order_Id, Cust_Id)
CUSTOMER      (Cust_Id, Cust_Name, Cust_City)
PRODUCT       (Product_Id, Product_Name, Unit_Price)
ORDER_ITEM    (Order_Id, Product_Id, Quantity)
```

Step 3 — Check for 3NF

In the CUSTOMER relation, Cust_Id → Cust_Name and Cust_Id → Cust_City are direct (non-transitive) dependencies on the primary key, so CUSTOMER is already in 3NF. Similarly, PRODUCT, ORDERS and ORDER_ITEM have no transitive dependencies, so all four relations satisfy 3NF (and in this case also BCNF, since Cust_Id, Product_Id and (Order_Id, Product_Id) are each the only candidate key of their relation).

This decomposition removes the insertion anomaly (a new product can now be added without needing an existing order to reference it), the deletion anomaly (deleting the last order of a customer no longer erases the customer's contact details), and the update anomaly (a customer's city now needs to be changed in only one row of CUSTOMER).

UNIT IV — Transaction Processing & Concurrency Control

4.1 Transaction and System Concepts

A transaction is a logical unit of database processing that includes one or more database access operations (read/insert/delete/update). It is the basic unit of recovery and concurrency control in a DBMS. A transaction must be executed atomically — either all its operations are completed successfully, or none of them are, to maintain database consistency even in the presence of failures.

States of a Transaction

A transaction moves through several states during its execution, tracked by the recovery manager:

- Active — the initial state; the transaction remains in this state while it is executing.
- Partially Committed — after the final statement has been executed, but before the changes are confirmed as permanently stored.
- Committed — after successful completion; changes are made permanent in the database.
- Failed — if the normal execution can no longer proceed due to hardware/logical errors.
- Aborted — after the transaction has been rolled back and the database is restored to its state prior to the start of the transaction.
- Terminated — the transaction leaves the system after commit or abort.

4.2 ACID Properties of Transactions

- Atomicity — a transaction is treated as a single, indivisible unit; either all of its operations are executed, or none are ('all or nothing').
- Consistency — a transaction takes the database from one consistent state to another, preserving all defined integrity constraints and rules.
- Isolation — the intermediate effects of a transaction are not visible to other concurrently executing transactions until it commits; concurrently running transactions appear to execute in isolation from one another.
- Durability (Permanency) — once a transaction commits, its changes must persist in the database even in the event of a system failure.

4.3 Schedules and Serializability

When multiple transactions execute concurrently, their operations become interleaved. The chronological order of execution of operations from various transactions is called a schedule (or history).

4.3.1 Serial and Concurrent Schedules

- Serial Schedule — a schedule in which the operations of each transaction are executed consecutively without any interleaving with operations of other transactions. Always consistent, but offers no concurrency (poor performance).

- Non-serial (Concurrent) Schedule — operations from multiple transactions are interleaved. May or may not preserve consistency, depending on how the operations are interleaved.

4.3.2 Serializability

A schedule is said to be serializable if it produces the same result as some serial execution of the same transactions — i.e., it is equivalent to a serial schedule. Serializability is used as the criterion for correctness of concurrent execution.

- Conflict Serializability — a schedule is conflict serializable if it can be transformed into a serial schedule by a series of swaps of non-conflicting operations. Two operations conflict if they belong to different transactions, access the same data item, and at least one of them is a write.
- View Serializability — a schedule is view serializable if it is view equivalent to some serial schedule (based on initial reads, updated reads, and final writes). Every conflict serializable schedule is view serializable, but the reverse is not always true.

Serializability is checked practically using a precedence graph (serialization graph): a directed graph with a node for each transaction, and an edge Ti → Tj if Tj reads or writes a data item after Ti has written it (a conflict exists where Ti's operation precedes Tj's). If the precedence graph contains no cycle, the schedule is conflict serializable.

4.4 Recoverability

Recoverability deals with the interaction of transaction commit and rollback (abort) with concurrency control, ensuring that no committed transaction ever has to be undone.

- Recoverable Schedule — a schedule where, for each pair of transactions Ti and Tj, if Tj reads a data item previously written by Ti, then the commit operation of Ti must appear before the commit operation of Tj.
- Cascadeless Schedule (avoids cascading rollback) — a schedule where every transaction reads only items that were written by already-committed transactions. This avoids the cascading rollback problem where the abort of one transaction forces the abort of several dependent transactions.
- Strict Schedule — a stricter condition where a transaction can neither read nor write a data item until the last transaction that wrote it has committed or aborted. Strict schedules simplify the recovery process and are cascadeless as well as recoverable.

4.5 Concurrency Control Techniques

Concurrency control protocols ensure that concurrent execution of transactions results in a consistent (serializable) database state, while still allowing multiple transactions to proceed simultaneously for good throughput.

4.5.1 Lock-Based Concurrency Control

A lock is a variable associated with a data item that describes the status of that item with respect to possible operations. Locking is the most widely used concurrency control mechanism.

Lock Type	Also Called	Description
Shared Lock (S)	Read Lock	Allows a transaction to read a data item; multiple transactions can hold shared locks on the same item simultaneously
Exclusive Lock (X)	Write Lock	Allows a transaction to both read and write a data item; only one transaction may hold an exclusive lock, and no other lock (S or X) can be held simultaneously

Two-Phase Locking Protocol (2PL)

A transaction follows the two-phase locking protocol if all locking operations precede the first unlock operation in the transaction. Every transaction is divided into two phases:

- Growing Phase — the transaction may acquire (obtain) new locks, but cannot release any.
- Shrinking Phase — the transaction may release existing locks, but cannot acquire any new ones.

It has been proven that if every transaction in a schedule follows the two-phase locking protocol, the schedule is guaranteed to be conflict serializable, eliminating the need to test for serializability explicitly. Variations include Strict 2PL (all exclusive locks held until commit/abort) and Rigorous 2PL (all locks — shared and exclusive — held until commit/abort).

Deadlock and Starvation

A major problem with locking is deadlock — two or more transactions are each waiting for locks held by the other(s), so none can proceed. Deadlocks are handled by deadlock prevention (e.g., wait-die and wound-wait schemes, which use transaction timestamps), deadlock detection using a wait-for graph, or timeouts. Starvation occurs when a particular transaction is repeatedly denied a lock due to a cycle of priorities; it can be avoided with a fair waiting scheme such as first-come-first-served lock granting.

4.5.2 Concurrency Control Based on Timestamp Ordering

A timestamp is a unique identifier created by the DBMS to identify a transaction, typically based on the system clock or a logical counter at the time the transaction starts. Timestamp-based protocols ensure serializability by choosing an ordering among transactions in advance, using their timestamps, without requiring locks.

Basic Timestamp Ordering (TO) Algorithm

For each data item X, the system maintains two timestamp values:

- read_TS(X) — the largest timestamp among all transactions that have successfully read X.
- write_TS(X) — the largest timestamp among all transactions that have successfully written X.

Rules applied whenever a transaction T (with timestamp TS(T)) issues a read or write on X:

- If T tries to write X, and TS(T) < read_TS(X) or TS(T) < write_TS(X), then T is attempting to write a value of X that was already read or written by a 'younger' transaction — T is rejected and rolled back (restarted with a new timestamp).
- If T tries to read X, and TS(T) < write_TS(X), then T is trying to read a value of X that was already overwritten — T is rejected and rolled back.

- Otherwise, the operation is executed and the appropriate read_TS(X) or write_TS(X) is updated to TS(T).

The basic timestamp ordering algorithm guarantees a conflict-serializable schedule, with the equivalent serial order being exactly the order of the transaction timestamps. Because rolled-back transactions are restarted with a new, larger timestamp, starvation should not occur.

Other Concurrency Control Approaches

- Multiversion Concurrency Control — keeps several versions of a data item, each tagged with the timestamp of the transaction that created it, so that read operations can be satisfied without waiting or aborting.
- Optimistic (Validation-based) Concurrency Control — transactions execute freely (read/write phase) and are validated for conflicts only just before commit (validation phase); if validation fails, the transaction is rolled back. Well suited to workloads with few conflicts.

4.5.3 Worked Example — Deadlock Detection using a Wait-For Graph

Suppose transaction T1 holds a lock on data item X and requests a lock on Y; simultaneously, transaction T2 holds a lock on Y and requests a lock on X. This is represented in a wait-for graph with nodes T1 and T2, and edges: T1 → T2 (T1 waits for T2) and T2 → T1 (T2 waits for T1). Since the graph contains a cycle (T1 → T2 → T1), the system is in deadlock.

Once detected (periodically, by the deadlock-detection component of the DBMS), the system resolves the deadlock by selecting a victim transaction (based on criteria such as least amount of work done, fewest locks held, or lowest priority) and rolling it back, releasing its locks so the other transaction can proceed.

Deadlock Prevention — Wait-Die and Wound-Wait Schemes

These schemes use transaction timestamps to decide whether a requesting transaction should wait or be rolled back, guaranteeing that no cycle (and hence no deadlock) can ever form:

Scheme	Rule when Ti requests a lock held by Tj
Wait-Die (non-preemptive)	If Ti is OLDER than Tj, Ti is allowed to WAIT. If Ti is YOUNGER than Tj, Ti is rolled back (DIES) and restarted later with the SAME timestamp.
Wound-Wait (preemptive)	If Ti is OLDER than Tj, Tj is rolled back (WOUNDED) and Ti proceeds. If Ti is YOUNGER than Tj, Ti WAITS.

In both schemes, a rolled-back transaction is restarted with its original timestamp preserved, which guarantees it eventually becomes the 'oldest' transaction and is no longer repeatedly aborted — thereby avoiding starvation.

4.6 Database Recovery Techniques (Overview)

Recovery techniques work closely with concurrency control to restore the database to a consistent state after a failure (system crash, transaction error, or media failure), guaranteeing the Atomicity and Durability properties of transactions.

4.6.1 Log-Based Recovery

The system maintains a log (journal) on stable storage, recording every update in the form <T, X, old_value, new_value> along with special records for transaction start, commit and abort. If a crash occurs, the log is used to redo the effects of committed transactions and undo the effects of incomplete ones.

- Deferred Update (NO-UNDO/REDO) — updates are recorded in the log but not applied to the actual database until the transaction commits; on crash, only REDO is needed for committed transactions, since uncommitted changes were never written to the database.
- Immediate Update (UNDO/REDO) — updates may be applied to the database as soon as they occur, even before commit; on crash, both UNDO (for uncommitted transactions) and REDO (for committed transactions) may be needed, using the before-images and after-images stored in the log.

4.6.2 Checkpoints

A checkpoint is a point of synchronization between the database and the log file, at which all buffers are force-written to disk. Checkpoints reduce recovery time because the recovery manager only needs to consider transactions that were active after the most recent checkpoint, rather than scanning the entire log from the beginning.

4.6.3 Shadow Paging

Shadow paging is an alternative to log-based recovery that avoids the need for UNDO/REDO altogether. The database is divided into fixed-size pages, referenced through a page table. Whenever a transaction updates a page, a new shadow copy of the page is created and the current page table is updated to point to it, while the original (shadow) page table remains unchanged. If the transaction commits, the current page table becomes the new page table; if it fails, the current page table is simply discarded, and the database reverts automatically to the pre-transaction (shadow) state.

4.7 Worked Example — Checking Serializability of a Schedule

Consider two transactions T1 and T2 operating on data items X and Y, interleaved as shown below:

Time	T1	T2
t1	read(X)	
t2	X = X + 100	
t3	write(X)	
t4		read(X)
t5		X = X * 1.1
t6		write(X)
t7	read(Y)	
t8		read(Y)

Here, T1's write(X) at t3 and T2's read(X) at t4 conflict (different transactions, same data item, one is a write). Since T1's write precedes T2's read, there is an edge T1 → T2 in the precedence graph. No other conflicting pair forces an edge in the opposite direction (T2 → T1), so the precedence graph has no cycle. Hence this schedule is conflict serializable and is equivalent to the serial schedule (T1, T2).

If, instead, T2 had written X before T1's write at t3 (creating both a T1 → T2 edge and a T2 → T1 edge), the precedence graph would contain a cycle, and the schedule would NOT be serializable — it would need to be rejected or re-ordered by the concurrency control manager.

Worked Example — Two-Phase Locking in Action

Applying strict 2PL to the same transactions: T1 first acquires an exclusive lock on X (growing phase), performs read(X) and write(X), then acquires a lock on Y for read(Y). T2 requests a lock on X but must wait until T1 releases it. Under strict 2PL, T1 holds all its exclusive locks until it commits, after which T2 is granted the lock on X and proceeds — this guarantees both serializability and recoverability, at the cost of T2 having to wait (reduced concurrency).

IMPORTANT QUESTIONS

The following question bank is organised unit-wise and mark-wise (1, 2, 5 and 8 marks) to help with quick revision as well as focused exam preparation.

Unit I — ER & EER Model: Important Questions

1 Mark Questions

1. Define a database and a DBMS.
2. What is an entity? Give an example.
3. Define a weak entity type.
4. What is a primary key?
5. Define cardinality ratio.
6. What is the degree of a relationship?
7. Define data independence.
8. What is a derived attribute? Give an example.
9. Name any two categories of database end users.
10. What is meant by generalization?

2 Marks Questions

11. Differentiate between a candidate key and a primary key.
12. Distinguish between total and partial participation constraints.
13. Differentiate between simple and composite attributes with examples.
14. What is the difference between an entity type and an entity set?
15. Differentiate between logical and physical data independence.
16. What is a multivalued attribute? How is it represented in an ER diagram?
17. Differentiate between specialization and generalization.
18. What are the three levels of the three-schema architecture?

5 Marks Questions

19. Explain the different types of attributes used in the ER model with suitable examples.
20. Describe the three-schema architecture of a DBMS with a neat diagram.
21. Explain the cardinality ratio and participation constraints in relationship types with examples.
22. What is a weak entity type? Explain with a suitable example, and describe how it is represented in an ER diagram.
23. Explain the various categories of database users and their roles.
24. Discuss the constraints on specialization and generalization (disjoint/overlapping and total/partial).

8 Marks Questions

25. Draw and explain an ER diagram for a University Database with at least four entities (STUDENT, COURSE, DEPARTMENT, INSTRUCTOR), showing attributes, keys and relationships with cardinality.
26. Explain the EER model concepts of specialization, generalization, aggregation and categories (union types) with suitable examples and diagrams.
27. Describe in detail the data models used in database systems and explain the concepts of schema and instance with examples.
28. Explain all ER diagram notations with their meaning, and construct an ER diagram for a Hospital Management System covering PATIENT, DOCTOR, APPOINTMENT and WARD.

Unit II — Relational Model & SQL: Important Questions

1 Mark Questions

29. Define a relation and a tuple.
30. What is a foreign key?
31. Name the categories of SQL commands.
32. What is the purpose of the GROUP BY clause?
33. Define referential integrity constraint.
34. What is the PROJECT operation in relational algebra?
35. Give the syntax of the DROP TABLE statement.
36. What is a natural join?
37. Define domain constraint.
38. What does the DISTINCT keyword do in SQL?

2 Marks Questions

39. Differentiate between DDL and DML with examples.
40. Differentiate between the WHERE clause and the HAVING clause.
41. Differentiate between DELETE and TRUNCATE commands.
42. What is the difference between a CANDIDATE KEY and a FOREIGN KEY?
43. Differentiate between the SELECT operation and the PROJECT operation in relational algebra.
44. What is the difference between an equi-join and a natural join?
45. List any four SQL aggregate functions with their purpose.
46. Differentiate between INNER JOIN and OUTER JOIN.

5 Marks Questions

47. Explain the relational integrity constraints with examples.
48. Explain the SELECT and PROJECT operations of relational algebra with suitable examples.
49. Explain the different types of JOIN operations in SQL with syntax and examples.

50. Write SQL statements to create a table EMPLOYEE with appropriate constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK), and explain each constraint.
51. Explain the DIVISION operation in relational algebra with a suitable example.
52. Explain the set operations (UNION, INTERSECTION, SET DIFFERENCE, CARTESIAN PRODUCT) used in relational algebra.

8 Marks Questions

53. Explain the Data Definition Language (DDL) and Data Manipulation Language (DML) statements of SQL with suitable syntax and examples for each command (CREATE, ALTER, DROP, INSERT, UPDATE, DELETE, SELECT).
54. For the relations EMPLOYEE(Emp_Id, Name, Dept_No, Salary) and DEPARTMENT(Dept_No, Dept_Name, Location), write SQL queries for: (a) list all employees earning more than 30000, (b) find the department-wise average salary, (c) list employee names with their department names using a join, (d) find employees who do not belong to any department.
55. Explain unary relational operations (SELECT, PROJECT, RENAME) and binary relational operations (JOIN, DIVISION) of relational algebra with examples.
56. Discuss the Relational Algebra and Relational Calculus with a comparison, and explain why relational algebra is called a procedural language.

Unit III — Normalization: Important Questions

1 Mark Questions

57. Define normalization.
58. What is a functional dependency?
59. Define 1NF.
60. What is a transitive dependency?
61. Define BCNF.
62. What is a partial dependency?
63. Define a multivalued dependency.
64. What is a trivial functional dependency?
65. Name any two update anomalies.
66. Define join dependency.

2 Marks Questions

67. Differentiate between partial dependency and transitive dependency.
68. Differentiate between 3NF and BCNF.
69. What is the difference between full functional dependency and partial functional dependency?
70. State Armstrong's Union and Decomposition rules.
71. Differentiate between insertion anomaly and deletion anomaly with examples.

72. Differentiate between 4NF and 5NF.

5 Marks Questions

73. Explain insertion, deletion and modification anomalies with suitable examples.
74. Explain Armstrong's inference rules (axioms) for functional dependencies.
75. Explain the First, Second and Third Normal Forms with suitable examples.
76. Explain BCNF and show, with an example, how a relation in 3NF may not be in BCNF.
77. Explain multivalued dependency and Fourth Normal Form with a suitable example.

8 Marks Questions

78. Define functional dependency. Explain the process of normalization from 1NF to BCNF using a suitable unnormalized relation, showing the decomposition at each step.
79. What is normalization? Explain 1NF, 2NF, 3NF and BCNF with suitable examples, and state the anomaly removed at each stage.
80. Explain Fourth Normal Form (4NF) and Fifth Normal Form (5NF) in detail with examples, including the concepts of multivalued dependency and join dependency.
81. Given the relation STUDENT_COURSE(Student_Id, Student_Name, Course_Id, Course_Name, Instructor, Grade) with appropriate functional dependencies, normalize it step-by-step up to 3NF, clearly stating the FDs and decomposition performed at each step.

Unit IV — Transaction Processing: Important Questions

1 Mark Questions

82. Define a transaction.
83. List the ACID properties.
84. What is a schedule?
85. Define serializability.
86. What is a shared lock?
87. Define deadlock.
88. What is a timestamp in concurrency control?
89. Define a cascadeless schedule.
90. What is meant by a serial schedule?
91. Define starvation.

2 Marks Questions

92. Differentiate between shared lock and exclusive lock.
93. Differentiate between conflict serializability and view serializability.
94. Differentiate between a recoverable schedule and a cascadeless schedule.
95. What is the difference between the growing phase and the shrinking phase in 2PL?

96. Differentiate between deadlock prevention and deadlock detection.
97. List the states of a transaction.

5 Marks Questions

98. Explain the ACID properties of a transaction with examples.
99. Explain the different states of a transaction with a state transition diagram.
100. Explain the concept of serializability and how a precedence graph is used to test conflict serializability.
101. Explain the Two-Phase Locking (2PL) protocol with its growing and shrinking phases.
102. Explain the basic timestamp ordering algorithm for concurrency control.

8 Marks Questions

103. Explain the ACID properties of a transaction in detail, and describe the various states through which a transaction passes during its execution.
104. What is concurrency control? Explain the Two-Phase Locking Protocol and its variations (Strict 2PL, Rigorous 2PL) with suitable examples. Also explain how deadlocks are handled.
105. Explain the timestamp-ordering based concurrency control technique in detail, including the rules applied for read and write operations, with a suitable example.
106. Differentiate between recoverable, cascadeless and strict schedules with examples, and explain why recoverability is important in transaction processing.

GLOSSARY OF IMPORTANT TERMS

Term	Definition
Attribute	A property or characteristic of an entity, represented as a column in a relation.
Candidate Key	A minimal set of attributes that can uniquely identify a tuple.
Cardinality	The number of tuples in a relation (also used for the maximum number of relationship instances an entity can participate in).
Data Independence	The capacity to change a schema at one level without affecting the schema at a higher level.
Deadlock	A situation where two or more transactions wait indefinitely for each other to release locks.
Degree	The number of attributes in a relation (also the number of participating entity types in a relationship).
Entity	A real-world object or concept about which data is stored.
Entity Integrity	The rule that no attribute participating in the primary key may be NULL.
Foreign Key	An attribute (or set of attributes) in one relation that references the primary key of another relation.
Functional Dependency	A constraint $X \rightarrow Y$ stating that the value of X uniquely determines the value of Y.

Term	Definition
Normalization	The process of decomposing relations to remove redundancy and anomalies.
Null Value	A special marker denoting a missing, unknown, or inapplicable attribute value.
Primary Key	The candidate key chosen to uniquely identify tuples in a relation.
Recoverability	A property ensuring that a schedule allows correct recovery after transaction failure.
Referential Integrity	The rule that a foreign key value must either be NULL or match an existing primary key value in the referenced relation.
Relation	A table consisting of rows (tuples) and columns (attributes).
Schema	The overall logical description/structure of a database.
Serializability	A correctness criterion stating that a concurrent schedule must be equivalent to some serial schedule.
Transaction	A logical unit of work comprising one or more database operations, executed atomically.
Weak Entity	An entity type without a key attribute of its own, identified via a relationship with an owner entity.

Text Books & Reference Books

Text Books

- Fundamentals of Database Systems — R. Elmasri, S. B. Navathe, Pearson Education.
- Database Management Systems — Rajiv Chopra, S. Chand Publications.

Reference Book

- An Introduction to Database Systems — C. J. Date, Pearson Education, New Delhi.